

Maze Runner

AI-Controlled Enemy — BFS vs A*
Complete Study Guide

Problem Statement 15

REVA University · B25CS0311 Portfolio Building

07/10/2026

github.com/Hyperval/maze-runner

Contents

1. Code Walkthrough
2. Viva Preparation
3. Project README
4. Member A — Core Logic
5. Member B — Maze & Balance
6. Member C — Interface & Testing
7. Member D — Analysis & Docs

Read chapters 1 and 2 before the viva — everyone, not just the person who wrote that part. Chapter 1 explains every file in plain language; chapter 2 is the expected questions with answers.

1. Code Walkthrough

Read this before the viva. It explains every file in plain language, assuming you know programming but not much Python. Work through it in order — each section builds on the last.

Total: about 1,100 lines across 8 files. None of it is magic; by the end of this document you should be able to point at any line and say what it does.

The 30-second version

The maze is a grid of 1s and 0s. The player and enemies each sit on one cell. Sixty times a second we read the keyboard, move everything one cell if its cooldown has expired, and redraw. The enemy decides where to move by running a search (BFS or A*) from itself to the player and stepping onto the second cell of the route it finds.

Everything else is detail on top of that.

Python things you need to know first

These five ideas cover ~90% of the syntax in this project.

1. Indentation defines blocks. There are no `{ }` braces. A line ending in `:` starts a block, and everything indented under it belongs to that block.

```
if x > 5:
    print("big")      # inside the if
print("always")      # outside it
```

2. Lists and tuples. A list `[1, 2, 3]` can be changed; a tuple `(1, 2)` cannot. We use tuples for coordinates — `(col, row)` — because a position shouldn't change in place, and because tuples can be dictionary keys.

3. Dictionaries map keys to values: `{"BFS": bfs_function}`. We use one to look up an algorithm by name, which is how `TAB` swaps them at runtime.

4. Classes bundle data with behaviour. `class Enemy:` is a blueprint. `Enemy(...)` makes one object from it. Inside the class, `self` means "this particular object", so two enemies each have their own `self.cell`.

5. List comprehensions build a list in one line:

```
[c for c in coins if c.cell != player.cell]
# means: a new list of every coin whose cell isn't the player's
```

We use this to drop a collected coin. Building a *new* list is safer than deleting from the one you're looping over, which silently skips items.

File 1: `settings.py` — all the numbers

Pure configuration. No logic. Every tunable value lives here so the rest of the code reads as logic rather than mystery numbers.

Two constants matter more than the rest:

```
PLAYER_MOVE_DELAY = 90    # ms between player steps
ENEMY_MIN_DELAY    = 140   # fastest the enemy can ever get
```

`ENEMY_MIN_DELAY` **must stay above** `PLAYER_MOVE_DELAY`. If the enemy's delay drops below the player's, the enemy moves more often than you do, and since it always walks a shortest path it catches you with mathematical certainty. Skill stops mattering. We originally had 85 here and `autoplay.py` proved the late levels were unwinnable.

Viva question: "how did you choose these numbers?"

We measured them. `autoplay.py` plays the game automatically hundreds of times and reports the win rate per level. We tuned until levels 1–3 sat around 70–90% and later levels around 30–40%.

File 2: `maze.py` — building the maze

The maze is a **list of lists**: `grid[row][col]`, where `1` is wall and `0` is floor. Note the order — row first, then column. Getting this backwards is the most common bug in grid code.

The carving algorithm (recursive backtracker)

1. Start at cell (1,1), mark it floor.
2. Look at neighbours TWO cells away that are still solid.
3. Pick one at random. Knock out the wall BETWEEN them. Move there.
4. If there are no unvisited neighbours, pop the stack and back up.
5. Done when the stack is empty.

Why two cells away? So walls always land on even coordinates and stay one cell thick. Carving one cell at a time would dissolve the grid into an open room with no walls at all.

Why must COLS and ROWS be odd? The carver only ever stands on odd coordinates. An even width would leave a ragged half-carved column at the edge.

`carve_steps()` and the `yield` keyword

```
def carve_steps(...):
    ...
    yield grid          # hand the grid back, then PAUSE here
```

A function containing `yield` is a **generator**. It doesn't run to completion — each `yield` hands a value to whoever is looping over it and freezes right there until the next iteration asks for more.

That's what lets the visualiser draw the maze being built one carve at a time without storing hundreds of copies. `generate()` just runs the generator to the end and keeps the final grid.

Braiding

A freshly carved maze is "**perfect**": exactly one route between any two cells, which means it is nothing but dead ends. The player gets cornered with zero counterplay.

`braid()` finds dead ends (cells with exactly one open side) and knocks out one more wall with 35% probability, creating loops and therefore escape routes.

This had a second effect we measured: loops also widen A*'s advantage over BFS, from ~10% to ~26%, because the search finally has real alternatives to discriminate between. See the benchmark table.

File 3: `pathfinding.py` — the technical core

This is the file examiners will press hardest on. Three functions.

`bfs(grid, start, goal)` — Breadth-First Search

```
queue = deque([start])          # a FIFO queue: first in, first out
seen = {start}                  # cells already queued, so we never revisit
came_from = {}                  # cell -> the cell we arrived from

while queue:
    current = queue.popleft()    # take the OLDEST cell
    if current == goal: ...done
    for nxt in neighbours(current):
        if nxt not in seen:
            seen.add(nxt)
            came_from[nxt] = current
            queue.append(nxt)
```

Why does this find the shortest path? Because it's a FIFO queue, every cell at distance 1 is processed before any cell at distance 2, which is processed before any cell at distance 3. So the first time you touch the goal, you got there in the fewest possible steps. That guarantee is a direct consequence of the queue being first-in-first-out — swap it for a stack and you get DFS, which finds *a* path, not the shortest.

Mark cells as seen when you ENQUEUE them, not when you dequeue them. Marking on dequeue lets the same cell get queued several times before it's processed, which wastes work and can loop forever.

Reconstructing the path: `came_from` is a trail of breadcrumbs. Start at the goal, follow the links back to the start, then reverse the list.

`astar(grid, start, goal)` — A*

Same idea, but instead of a plain queue it uses a **priority queue** (a heap), and always expands the cell with the lowest score:

```
f = g + h
g = steps actually taken to reach this cell
h = Manhattan distance remaining to the goal (|dx| + |dy|)
```

So A* leans toward the goal instead of spreading out evenly in all directions.

Why must `h` never overestimate? If it did, A* could skip the cell that's actually on the shortest path because it looks worse than it really is, and you lose the optimality guarantee. Manhattan distance on a 4-direction grid can never overestimate, because the real walk is at least that long and usually longer. A heuristic with this property is called **admissible**.

The `counter` in the heap entries: entries are `(f, counter, cell)`. If two cells tie on `f`, Python compares the next item. Tuples of ints compare fine; comparing the `cell` tuples would be arbitrary. The counter guarantees a tie is broken before it ever reaches the cell.

`flood_distances(grid, start)`

BFS with no goal — it just keeps expanding until it has measured the whole reachable region, returning `{cell: distance}` for every cell.

We use it to place enemies and coins. **Why not just use Manhattan distance?** Because Manhattan ignores walls: it will call two cells "adjacent" when a wall between them makes the real walk 40 steps.

File 4: `entities.py` — player, enemy, coin

Why movement is timed in milliseconds

```
if now - self._last_move < self.move_delay:
    return False          # cooldown still running, refuse the move
```

If we counted *frames* instead, the game would run faster on a better laptop. Timing in milliseconds makes it behave identically everywhere.

How the enemy actually moves

```
self.recompute_path(grid, player_cell)    # search from me to the player
if len(self.path) >= 2:
    self.cell = self.path[1]              # step onto the SECOND cell
    self.path = self.path[1:]
```

Why `path[1]` and not `path[0]`? Because `path[0]` is the cell the enemy is already standing on. The next step is index 1.

Why recompute every step? The player keeps moving. A path computed once would be chasing where they *were*. At ~650 cells a search takes a fraction of a millisecond, so recomputing is cheap.

Each `Enemy` object stores its own `algorithm`, so creating one with `"BFS"` and another with `"A*"` makes them hunt independently — that's the whole two-enemy feature, and it needed no new pathfinding code at all.

File 5: `main.py` — the game itself

The loop

```
while True:
    self.handle_events()  # read keyboard/window events
    self.update()         # move everything, decide win/loss
    self.draw()           # paint the frame
    self.clock.tick(FPS)  # sleep so we run at 60fps, not 5000fps
```

Every real-time game is this loop. Everything else hangs off it.

The state machine

```
MENU, PLAYING, PAUSED, WON, LOST = "menu", "playing", "paused", "won", "lost"
```

`self.state` holds exactly one of these. `update()` returns immediately unless the state is `PLAYING`, which is what makes pausing work — nothing moves, but drawing continues, so the frozen frame stays on screen.

Using strings rather than numbers means debug output reads `"playing"` instead of `2`.

Enemy placement — the bug worth knowing

Our first version put the enemy on the cell farthest from the player. That's the opposite corner — **which is exactly where the exit is**. The enemy sat on the goal and camped it. Level unwinnable.

```
def score(cell):
    return min(from_player[cell], from_exit[cell])
first_cell = max(cells, key=score)
```

Maximising the **smaller** of the two distances forces a spawn well away from both. The second enemy additionally maximises distance from the first, or the two walk in lockstep and the comparison is invisible.

Keeping the pause clock honest

`pygame.time.get_ticks()` keeps counting while paused. If we ignored that, the timer would jump forward the moment you resumed. So we record when the pause began, add the duration to `paused_total`, and subtract it:

```
self.elapsed = (now - self.start_time - self.paused_total) / 1000.0
```

Why the window is a fixed size

`WIDTH / HEIGHT` are computed once from `MAX_COLS / MAX_ROWS`. Levels grow the maze but it's capped at those maxima, and smaller mazes are centred using `offset_x / offset_y`. If the maze could outgrow the window, it would be drawn off-screen.

File 6: `benchmark.py` — measuring BFS vs A*

Standalone: it imports the game's modules but never modifies them, so it cannot break the demo.

Two Python details worth knowing:

```
matplotlib.use("Agg")    # BEFORE importing pyplot
```

"Agg" draws to a file instead of opening a window. Without it the script blocks forever waiting for you to close a plot.

```
begin = time.perf_counter()
for _ in range(20): search(...)    # repeat, because one run is too fast
```

`perf_counter` is the high-resolution clock. A single search is faster than the clock's resolution, so we time 20 and divide.

What it found

Braid	BFS cells	A* cells	A* saves	BFS ms	A* ms	Path
0.00	183.5	163.9	10.7%	0.141	0.216	130.2
0.12	188.0	160.8	14.5%	0.145	0.212	108.5
0.35	214.1	157.9	26.2%	0.169	0.219	87.5
0.50	236.1	176.0	25.5%	0.212	0.279	81.2
0.80	256.3	171.9	33.0%	0.250	0.302	65.5

Two findings, and the second one is the interesting one.

1. A*'s advantage depends on how open the maze is. In a tight maze it saves only ~11%, because the corridors are so constrained there's barely any choice for the heuristic to discriminate between. Open it up with loops and the saving climbs past 30%.

2. A* expands fewer cells but takes MORE wall-clock time. 0.219ms vs 0.169ms at our settings. This looks like a contradiction and it is not:

- BFS uses a `deque`. Adding and removing is $O(1)$ — a couple of pointer moves.
- A* uses a **heap**. Every push and pop is $O(\log n)$, and it computes the Manhattan heuristic for every neighbour on top.

So A* does *less* work in the sense of touching fewer cells, but *more* work per cell. At 650 cells the per-cell overhead dominates. On a much larger map, or where visiting a cell is expensive (reading from disk, querying a server), A* wins decisively.

This is an excellent viva answer — it shows you measured rather than assumed, and that you understand the difference between an algorithm's complexity and its real-world constant factors.

File 7: `autoplay.py` — the robot tester

Runs the game headlessly with a bot playing it, hundreds of times, hunting for crashes and unwinnable levels.

```
os.environ["SDL_VIDEODRIVER"] = "dummy"    # BEFORE importing pygame
```

Tells SDL to render to nothing, so no window opens and runs finish in seconds.

The bot walks the shortest route to its objective (nearest coin, then the exit) using the same BFS the enemy uses, and **routes around the enemies** by treating the cells near them as temporary walls:

```
safe_grid = [row[:] for row in game.grid]    # copy each row
```

`row[:]` makes a copy of the row. A plain `list(grid)` would copy the outer list but share the inner row lists — so writing to `safe_grid` would corrupt the real maze. This mutable-aliasing trap catches everyone once.

What it found — the headline result

The first run reported **levels 3–10 unwinnable, 0% win rate**. Investigating in stages:

1. The bot was ignoring enemies and walking straight into them. Added evasion → still only 0–20%. So the bot wasn't the whole story.
2. Coins were spawning in the farthest cells, so every coin meant crossing the entire maze under pursuit. Moved them to a middle distance band and added a 2.2-second head start → levels 1–3 reached 47–73%.
3. Levels 4+ still collapsed — exactly where the second enemy joins. Slowing it by 1.45× helped only slightly, so that wasn't the root cause either.
4. **Real cause:** `ENEMY_MIN_DELAY` was 100ms against the player's 90ms — only 1.1× faster than something that never rests and always takes the shortest route. Raising it to 140ms fixed the curve.

Final: L1 73%, L2 93%, L3 80%, later levels 20–40%.

The honest framing for the viva: the bot is a **floor, not a ceiling**. It only dodges one cell ahead and never plans a route that keeps the enemy at distance, so a human who learns a level will beat these numbers.

File 8: `test_pathfinding.py` — 210 checks

Run after **every** change:

```
python test_pathfinding.py
```

The most valuable checks:

Check	The bug it guards against
Every cell reachable, 20 mazes	A maze where the enemy can't reach you — chase silently breaks
A* path length == BFS path length	A broken heuristic that finds a non-optimal route
A* expands ≤ BFS	A heuristic that actively makes things worse
Enemy > 8 steps from exit	The exit-camping bug
<code>ENEMY_MIN_DELAY > PLAYER_MOVE_DELAY</code>	The enemy becoming strictly faster than you
300 random moves + rendering	Any crash from an input nobody thought of

The last one is a **fuzz test** — it does random things and only asserts "don't crash". It finds the bug that would otherwise appear in front of the examiner.

If you only memorise five things

1. **BFS uses a FIFO queue, so it finds the shortest path.** Everything at distance d is finished before anything at $d+1$ is touched.
 2. **A* scores cells** $f = g + h$, where h is Manhattan distance and must never overestimate, or optimality is lost.
 3. **A* expanded ~26% fewer cells but ran slower in wall-clock time**, because its heap costs $O(\log n)$ per operation against the deque's $O(1)$.
 4. **The enemy spawn bug:** "farthest from the player" is the exit corner, so it camped the goal. Fixed by maximising `min(dist_to_player, dist_to_exit)`.
 5. **We tuned difficulty by measurement, not feel** — `autoplay.py` ran hundreds of bot playthroughs and found the enemy was only $1.1\times$ slower than the player.
-

Suggested reading order

1. `settings.py` — 5 minutes, it's just numbers
2. `maze.py` → `generate()` and `braid()`
3. `pathfinding.py` → `bfs()` first, then `astar()`
4. `entities.py` → `Enemy.update()`
5. `main.py` → `run()`, then `update()`, then `new_level()`
6. The two tools last — they're independent of the game

Each person should be able to explain their own file **and** `pathfinding.py`, because that's the one that gets asked about.

2. Viva Preparation

Problem Statement #15 — Maze Runner with AI-Controlled Enemy

Worth protecting: **Problem Understanding & Explanation (4 pts)** directly, and **Technical Implementation (6 pts)** indirectly — because an examiner who suspects you can't explain your code marks the technical section down too.

The three rules

1. **Never say "I don't know" and stop.** Say what you *do* know, then say what you'd check. "I didn't write that part, but it works by X — Rahul owns it and can give the detail" is a fine answer. Silence is not.
 2. **Answer only what's asked.** Volunteering extra invites questions you haven't prepared for.
 3. **If you don't understand the question, ask them to repeat it.** Guessing at a misheard question looks worse than asking.
-

Tier 1 — Everyone must be able to answer these

"What does your project do?"

A maze game where the player has to reach the exit while an enemy chases them using pathfinding. The enemy recalculates the shortest route to the player on every step, using either BFS or A*, and you can switch between them live and watch the difference.

Practise this until it's 15 seconds and automatic. It's the first question every time.

"What's the purple area on screen?"

Those are the cells the search algorithm expanded while looking for the player. It makes the AI visible instead of invisible — you can see BFS spread out evenly in all directions, while A* leans toward the player.

"Why BFS? Why not something else?"

On a grid where every move costs the same, BFS is guaranteed to find the shortest path, and it's simple enough that we could implement and verify it ourselves. We added A* alongside it so we could

compare.

"What's your tech stack?"

Python with pygame-ce for graphics and input. No external datasets, no network, no hardware — everything is generated at runtime.

"What would you do differently with more time?"

Pick **one** and be specific. Good answers:

An enemy that predicts where the player is heading instead of chasing where they currently are, so it can cut them off at a junction.

Dijkstra with weighted terrain — mud cells costing more to cross — which would show why BFS is really just Dijkstra with all edge costs equal.

Bad answer: "add more levels and better graphics."

Tier 2 — Algorithms (Member A leads, everyone should follow)

"How does BFS guarantee the shortest path?"

It uses a FIFO queue, so it finishes exploring everything at distance 1 before it touches anything at distance 2, and so on. The first time it reaches the goal, it must have got there by the fewest possible steps.

"What is A*, and how is it different?"

A* scores each cell as $f = g + h$. g is the steps actually taken to reach it, h is an estimate of the steps remaining — we use Manhattan distance. It always expands the lowest- f cell, so instead of spreading out evenly it leans toward the goal. It finds the same shortest path, but usually expands fewer cells.

"Why must the heuristic never overestimate?"

If it overestimates, A* might skip the cell that's actually on the shortest path because it looks worse than it is, and you lose the optimality guarantee. Manhattan distance on a grid with no diagonal movement can never overestimate, because the real path is at least that long and usually longer.

"Which is better, BFS or A*?"

This is a trap. The honest answer scores better:

It depends on the maze. We measured it across 20 generated mazes. In a tight maze with no loops, A* only expanded about 9% fewer cells than BFS, because the corridors are so constrained there's barely any choice for the heuristic to discriminate between. Once we opened dead ends into loops, A*'s advantage rose to about 30%. So A* helps most when there are real alternative routes.

Have the README table open on screen when you say this. Measured evidence is rare in a hackathon and examiners notice it.

"What's the time complexity?"

$O(V + E)$ for BFS, where V is the number of cells and E the connections between them. On a grid each cell has at most 4 neighbours, so E is about $4V$, which makes it $O(\text{number of cells})$. Our maze is 31×21 , so roughly 650 cells — fast enough to recompute every single step.

"Why recompute the path every step instead of once?"

Because the player keeps moving. A path computed once would be chasing where the player *was*. It's cheap enough at this maze size to just recompute.

"Why no diagonal movement?"

The walls are drawn on the grid, so a diagonal move would let the enemy slip through the corner point where two walls meet. Restricting to four directions keeps the enemy physically inside the corridors.

Tier 3 — Maze generation (Member B leads)

"How do you generate the maze?"

A recursive backtracker. Start at a cell, pick a random unvisited neighbour two cells away, knock out the wall between them, move there, repeat. When there's nowhere to go, back up the stack. It's depth-first search used to carve instead of to search.

"Why two cells away?"

So that walls always land on even coordinates and stay one cell thick. If we carved one cell at a time, the whole grid would dissolve into an open room.

"What's braiding?"

A plain generated maze is 'perfect' — exactly one route between any two cells, which means it's all dead ends. The player gets cornered with no way out and it isn't fun. We open about 35% of dead ends into loops so there are escape routes. It also made the BFS-vs-A* difference bigger, because now there are genuinely alternative routes to choose between.

"How did you pick the difficulty numbers?"

We playtested. [Show the playtest table.] The key constraint is that the enemy's move delay has to stay above the player's, or it catches you with certainty and skill stops mattering.

Tier 4 — Implementation (Member C leads)

"Walk me through the game loop."

Handle events, update game state, draw the frame, tick the clock to cap the frame rate. Standard pygame structure.

"Why milliseconds instead of frame counts for movement?"

So the game plays identically on a fast and a slow machine. If we counted frames, the enemy would be faster on a better laptop.

"How do you detect collisions?"

Everything moves one whole cell at a time on the grid, so a collision is just checking whether two things are on the same cell. No rectangle overlap maths needed.

"How did you test it?"

A headless test suite — 87 checks that run without opening a window. It verifies every cell in the maze is reachable, that paths never cross walls, that A* always finds the same length path as BFS while expanding no more cells, and that the enemy never spawns on the exit.

Tier 5 — The hard ones

"Did you write this yourselves?"

Be honest and specific. Confidence comes from detail:

We used AI assistance for the initial scaffolding, then each of us took ownership of a module. I rewrote the BFS implementation myself to make sure I understood it — [show `scratch_bfs.py`] — and I can walk you through any part of this file.

Then *actually do it* if asked. This is exactly why Member A's Part 1 exists.

"What was the hardest bug?"

The enemy spawning on the exit. We placed it at the cell farthest from the player, which is the opposite corner — exactly where the exit is. It camped the goal and the level was impossible. We fixed it by maximising the smaller of the distance to the player and the distance to the exit, using real walking distance instead of straight-line distance so walls are counted.

"What's the weakest part of your project?"

Don't say "nothing." Pick something real and show you understand it:

The Manhattan heuristic isn't great in a twisty maze — when the real route winds a lot, straight-line distance is a poor guide and A* ends up expanding almost as much as BFS. A better heuristic would account for the maze structure, but computing one would cost more than it saves.

"Can it handle a bigger maze?"

Yes — the maze size is a constant in settings. BFS is linear in the number of cells, so a maze four times bigger takes about four times longer to search, which is still well under a frame. The real limit is the window size.

"What happens if the enemy can't reach the player?"

It can't happen in our mazes — the generator guarantees every cell is connected, and we test that across 20 generated mazes. But the code handles it anyway: the search returns an empty path and the enemy just doesn't move, rather than crashing.

Demo script (3–4 minutes, rehearse it twice)

Time	Do	Say
0:00	Title screen	"Problem statement 15 — maze game with a pathfinding enemy."
0:20	Start level 1, move around	"Player reaches the exit, enemy chases."
0:45	Press V to show overlay	"Purple is what the search explored, red is the route it chose."
1:15	Press TAB to A*	"Same situation under A* — notice fewer cells expanded, same path length."
1:45	Point at HUD counter	"That number is the cells expanded — it's the comparison, live."
2:15	Reach the exit, win	"Next level, the enemy gets faster."
2:45	Show the README table	"We measured this across 20 mazes — here's where A* actually helps."
3:15	Close	"Future scope: a predictive enemy that cuts you off."

Rules for the demo - Play on a level you've practised. Don't improvise. - If it crashes: *don't panic, don't apologise repeatedly*. Say "let me show you the recorded demo" and open the video. Having it ready is worth more than it never crashing. - Whoever presents each section should be the person who owns that module.

Final 10 minutes before the viva

- Game already running, on the title screen
- README open in a second window at the measured-comparison table
- Backup video open in a third window, ready to play
- Phone on silent
- Decide who answers first — one person leads, others add detail when asked

3. Project README

REVA University — B25CS0311 Portfolio Building Problem Statement #15 · 07/10/2026

Problem Statement

Build a maze game where the player navigates to an exit while an enemy character chases them using a basic pathfinding algorithm (BFS or A*).

Approach

The maze is generated at runtime with a recursive-backtracker carver, then "braided" to open some dead ends into loops so the player has escape routes. Each enemy recomputes a route to the player's current cell on every step using either BFS or A*, and walks one cell along it. Both searches expose the cells they expanded, which the game draws as a translucent overlay — so the AI's reasoning is visible on screen rather than hidden.

From level 4 a **second enemy** joins running the *other* algorithm, so BFS and A* hunt the same player simultaneously and the difference in how much of the maze each one searches is visible in a single frame.

Tech Stack

Component	Choice
Language	Python 3.10+
Graphics / input	pygame-ce 2.5.8
Algorithms	BFS, A* (Manhattan heuristic), recursive-backtracker generation
Testing	Custom headless test suite (<code>test_pathfinding.py</code>)

No external datasets, no network, no hardware.

How to Run

```
pip install -r requirements.txt
python main.py
```

Run the tests (headless, no window opens):

```
python test_pathfinding.py
```

Controls

Key	Action
Arrow keys / WASD	Move
SPACE	Start / next level / retry
1 / 2	Put the enemy on BFS / A* directly
TAB	Swap the two algorithms
V	Toggle search visualisation
P	Pause
R	Restart level
M	Back to menu
ESC	Quit

Collect every coin to unlock the exit.

Project Structure

```
maze-runner/  
├─ main.py           Game loop, rendering, input, state machine  
├─ pathfinding.py    BFS, A*, flood-fill distance maps  
├─ maze.py           Maze generation (incl. step-by-step carving)  
├─ entities.py       Player, Enemy and Coin  
├─ settings.py       All tunable constants  
├─ visualise_maze.py Watch the maze being carved, step by step  
├─ benchmark.py      Measures BFS vs A*, writes assets/benchmark.png  
├─ autoplay.py       Headless bot playthroughs; finds crashes + bad balance  
├─ tools_screenshot.py Generates the demo screenshots  
├─ test_pathfinding.py Headless test suite (210 checks)  
├─ docs/             Per-member guides, code walkthrough, viva prep  
└─ assets/           Demo screenshots and the benchmark chart
```

New to the codebase? Read [docs/CODE_WALKTHROUGH.md](#) — every file explained in plain language, with the Python concepts you need.

How the Enemy Works

Each time the enemy is due to move:

1. Run the selected search from the enemy's cell to the player's cell.
2. The search returns the route **and** the list of cells it expanded.
3. The enemy steps to the second cell in that route (the first is where it already stands).

4. The renderer draws the expanded cells in purple and the chosen route in red.

BFS uses a FIFO queue, so it finishes every cell at distance d before touching distance $d+1$. The first time it reaches the goal, it got there by a shortest route — guaranteed.

A* scores each cell $f = g + h$, where g is steps taken so far and h is the Manhattan distance left to the goal. Because the heuristic never overestimates, A* finds the same shortest path, but it leans toward the goal instead of spreading evenly.

Measured: does A* actually help?

Produced by `benchmark.py`, averaged over **50 generated mazes** per row:

Braid chance	BFS cells	A* cells	A* saving	BFS ms	A* ms	Avg path
0.00	183.5	163.9	10.7%	0.141	0.216	130.2
0.12	188.0	160.8	14.5%	0.145	0.212	108.5
0.35	214.1	157.9	26.2%	0.169	0.219	87.5
0.50	236.1	176.0	25.5%	0.212	0.279	81.2
0.80	256.3	171.9	33.0%	0.250	0.302	65.5

A* expands fewer cells but takes more wall-clock time. That is not a contradiction: BFS uses a deque ($O(1)$ per push/pop) while A* uses a heap ($O(\log n)$) and computes a heuristic for every neighbour. At ~650 cells the per-cell overhead outweighs the cells saved. On a much larger map, or where visiting a cell is expensive, A* wins decisively.

The interesting result is that **A*'s advantage depends on how open the maze is**. In a tight, perfect maze (braid 0.00) the corridors are so constrained that there are barely any choices to make, and A* saves only ~9%. As loops are added the search has real alternatives, the Manhattan heuristic starts to discriminate between them, and the saving climbs past 30%.

A single frame can look worse than the average: when the route winds heavily, straight-line distance is a poor guide and A* expands almost as much as BFS. This is the honest limitation of the Manhattan heuristic in a twisty maze.

We ship with braid chance **0.35** — it keeps the A* advantage visible *and* gives the player escape routes, which makes the chase fairer.

Design Decisions Worth Noting

Enemy placement. Spawning the enemy at the cell farthest from the player puts it in the opposite corner — which is exactly where the exit is, so it camps the goal and the level becomes unwinnable. We instead maximise the *smaller* of (distance to player, distance to exit), using true walking distances rather than Manhattan, so walls are respected.

Braided maze. A perfect maze is all dead ends, so the player gets cornered with no counterplay. Opening ~35% of dead ends creates loops to escape through.

Grid-aligned movement. Entities move one whole cell at a time rather than by pixels. Collision becomes an equality check, and the pathfinding result maps directly onto movement with no interpolation.

Time-based movement. Move cooldowns are in milliseconds, not frames, so the game plays identically regardless of frame rate.

Testing

`test_pathfinding.py` runs 87 headless checks covering:

- Maze generation: solid border, walkable corners, seed reproducibility
- Connectivity: every floor cell reachable, across 20 generated mazes
- Path legality: correct endpoints, one-step moves, never crosses a wall
- A* optimality: path length always equals BFS, expands no more cells
- Edge cases: start equals goal, unreachable goal returns empty rather than hanging
- Enemy spawn: never camping the exit, never on top of the player

Screenshots

<code>assets/demo_BFS.png</code>	BFS chase with search overlay
<code>assets/demo_Astar.png</code>	Same position under A*
<code>assets/demo_win.png</code>	Win state

Future Scope

- Multiple enemies running different algorithms simultaneously
- Dijkstra with weighted terrain (mud slows movement) to show why uniform-cost search generalises BFS
- Enemy that predicts where the player is heading instead of chasing the current cell
- Fog of war limiting player vision radius
- Replay export of a chase for analysis

Team

Role	Member	Contribution
A — Core logic	<i>(name, roll no.)</i>	Pathfinding (BFS, A*, distance maps)
B — Supporting logic	<i>(name, roll no.)</i>	Maze generation, entity movement
C — Interface / testing	<i>(name, roll no.)</i>	Rendering, HUD, search overlay, test suite
D — Docs / presentation	<i>(name, roll no.)</i>	README, slides, demo recording

4. Member A — Core Logic

You own `pathfinding.py`. This is the technical heart of the project and the part judges will question hardest. Your job is split in two: *be able to explain it*, then *extend it*.

Part 1 (do this first, ~45 min): Rebuild BFS from memory

This is not busywork. In the viva, someone will point at the screen and ask "how does the enemy decide where to go?" If you can rebuild BFS yourself, that question is free marks. If you can't, we lose points on

Technical Implementation (6) and **Problem Understanding (4)**.

Steps

1. Read `pathfinding.py`, specifically `bfs()`. Take your time.
2. Close it. Open a new file `scratch_bfs.py`.
3. Write BFS from scratch. You need: - a queue of cells still to explore (`collections.deque`) - a set of cells you've already seen, so you never revisit - a dict `came_from` mapping each cell to the cell you arrived from
4. Test it against the real one:

```
import maze
from pathfinding import bfs
from scratch_bfs import my_bfs

g = maze.generate(seed=1)
mine, _ = my_bfs(g, (1, 1), (29, 19))
theirs, _ = bfs(g, (1, 1), (29, 19))
print(len(mine), len(theirs)) # must match
```

If you get stuck

- **Infinite loop?** You're not marking cells as seen *when you add them to the queue*. Mark on enqueue, not on dequeue, or the same cell gets queued twice.
- **Path comes out backwards?** That's expected — you follow `came_from` from the goal back to the start, so reverse the list at the end.
- **KeyError in reconstruction?** The goal was never reached. Return `[]`.

Checkpoint

You can explain, without notes: *why does BFS find the shortest path?* (Answer: it's a FIFO queue, so every cell at distance 1 is finished before any cell at distance 2 is touched. The first time you reach the goal, you got there by the fewest steps possible.)

Part 2 (~2 hours): Add a second enemy

Right now there's one enemy. Two enemies running **different** algorithms at once is the single best Innovation feature (3 pts) — judges can watch BFS and A* hunt the same player simultaneously and see the difference live.

Steps

1. In `main.py`, find `new_level()`. It creates `self.enemy = Enemy(...)`. Change it to a list: `self.enemies = [Enemy(...), Enemy(...)]`.
2. Give the second enemy `algorithm="A*"` and a different start cell. Reuse the existing `flood_distances` logic — pick the *second* best spawn, or just a cell far from both the first enemy and the player.
3. In `update()`, loop over `self.enemies` instead of touching one.
4. In `draw_entities()`, loop and draw each. Give them different colours — add `C_ENEMY_2` to `settings.py`.
5. The catch check becomes: caught if **any** enemy is on the player's cell.
6. In `draw_search()`, decide: draw both overlays (messy) or only the first enemy's (cleaner). Recommend a key to cycle which enemy's search is shown.

Watch out for

- `draw_hud()` reads `self.enemy.algorithm` and `self.enemy.last_search_size`. Update it or it'll crash with `AttributeError`.
- `handle_events()` TAB handler also references `self.enemy`.
- `test_pathfinding.py` has an enemy-spawn block referencing `game.enemy`. Member C will need to update it — tell them when you change this.

Make it safe

Keep the second enemy **behind a toggle** (a key, or a constant in `settings.py`). If it misbehaves during the demo, you flip it off and still have a working single-enemy game. Never let a new feature be able to break the demo.

Checkpoint

Two enemies chase, each with its own colour, and the game still wins/loses correctly. Run `python test_pathfinding.py` — it must still pass.

Stretch (only if Part 2 is solid and tested)

Add **Dijkstra** with weighted terrain: mark some cells as "mud" costing 3 steps instead of 1. This shows why BFS is just Dijkstra where every edge costs the same — a genuinely strong point to make in the viva.

What you must be able to answer

- How does BFS guarantee the shortest path?
- What is the heuristic in A^* , and why must it never overestimate?
- Why no diagonal movement? (It would let the enemy cut through wall corners.)
- Why recompute the path every step instead of once?
- What's the time complexity? ($O(V+E)$; here $V = \text{cells}$, $E \approx 4V$, so $O(\text{cells})$.)

5. Member B — Maze & Balance

You own `maze.py` and the tuning in `settings.py`.

Your work needs the least new code and has the most direct effect on whether the game is *fun* — which is what the judges actually experience during the demo. Take it seriously: a technically perfect game that's unplayable scores badly on **Functionality (8)**.

Part 1 (~45 min): Understand the generator, then prove you do

Read `maze.py`. The algorithm is a **recursive backtracker**:

1. Start at cell (1,1), mark it floor.
2. Look at neighbours **two cells away** that are still solid wall.
3. Pick one at random, knock out the wall *between* the two cells, move there.
4. No unvisited neighbours? Pop the stack and back up.
5. Stack empty means every cell has been visited.

Why two cells away? Carving at odd coordinates only means even coordinates always stay as walls, giving clean one-cell-thick walls. Carving one cell at a time would dissolve the whole grid into an open room.

Prove you understand it

Add a `--watch` mode that draws the maze as it's being carved, one step at a time. ~20 lines, looks great in the demo, and you can't write it without understanding the stack. **This is your Innovation contribution.**

Hint: make `generate()` a generator function that `yield`s the grid after each carve step, then have a small pygame loop draw each yielded frame.

Part 2 (~1.5 hours): Difficulty curve

Right now every level is the same 31×21 maze with a slightly faster enemy. That's thin. Your job is to make levels 1→5 feel like a real progression.

What you can tune — all in `settings.py`

Constant	Now	Effect
<code>COLS</code> , <code>ROWS</code>	31, 21	Maze size. Must be odd numbers.
<code>PLAYER_MOVE_DELAY</code>	90 ms	Lower = player moves faster
<code>ENEMY_MOVE_DELAY</code>	190 ms	Lower = enemy faster = harder
<code>ENEMY_SPEEDUP_PER_LEVEL</code>	18 ms	How fast difficulty ramps
<code>ENEMY_MIN_DELAY</code>	85 ms	Floor, so it never becomes impossible
<code>BRAID_CHANCE</code>	0.35	Fraction of dead ends opened into loops

The key relationship

If `ENEMY_MOVE_DELAY` drops below `PLAYER_MOVE_DELAY` (90), the enemy is **faster than the player** and, because it always takes the shortest path, it catches you with certainty. The game stops being winnable by skill.

`ENEMY_MIN_DELAY = 85` is below 90 — so at max level the enemy *is* faster. **Decide if that's what you want.** Either raise the floor to ~100, or keep it and give the player an escape mechanic. Your call, but make it deliberate and be ready to explain it.

Make maze size scale with level

In `main.py` 's `new_level()` , `maze.generate()` is called with no arguments. Pass bigger dimensions as level rises:

```
cols = min(41, COLS + (self.level - 1) * 2)  # keep it ODD
rows = min(27, ROWS + (self.level - 1) * 2)
```

Watch out: `WIDTH` and `HEIGHT` in `settings.py` are computed from `COLS` / `ROWS` at import time. If the maze grows past the window, it gets cut off. Either cap the growth (as above) or resize the window — capping is safer.

Testing method

Play each level 3 times and record it:

Level	Won?	Time	Felt
1			too easy / right / too hard

Target: level 1 winnable first try, level 5 takes a few attempts. Put this table in the slides — "we playtested and tuned" is exactly the kind of evidence that earns **Problem Understanding (4)**.

What you must be able to answer

- Why does the carver step two cells at a time?
- What is a "perfect" maze, and why did we deliberately break it with braiding? (Perfect = exactly one route between any two cells = all dead ends = player gets cornered with no counterplay.)
- Why must COLS and ROWS be odd?
- How did you choose the difficulty numbers? (Point at your playtest table.)
- What happens if the enemy is faster than the player?

6. Member C — Interface & Testing

You own what the judges actually see, plus `test_pathfinding.py`.

The rubric gives **Presentation & Demo (2)** and part of **Functionality (8)** to things in your lane. Also: you own "does this break when I do something weird", which is what loses teams marks at the worst moment.

Part 1 (~1 hour): Menu & game-state screens

Right now the game drops you straight into level 1, and the only way out is ESC. Add a proper front end.

What to build

1. **Title screen** — game name, "Press SPACE to start", controls list.
2. **Pause** — P key freezes everything and shows an overlay.
3. **Game over** — already exists as a banner; make it show best time per level.

How

`main.py` already has a state machine: `PLAYING`, `WON`, `LOST` (see the constants near the top, and `self.state`). You're adding `MENU` and `PAUSED`.

- In `update()`, return early unless `self.state == PLAYING` (it already does this — extend the idea).
- In `draw()`, branch: if `MENU`, call a new `draw_menu()` and skip the maze.
- In `handle_events()`, SPACE on `MENU` starts the game; P toggles `PAUSED`.

Watch out

`self.start_time` is set in `new_level()` and `elapsed` is computed from `pygame.time.get_ticks()`. If you pause, the clock keeps running and the timer jumps. Fix: track accumulated paused time and subtract it.

Part 2 (~45 min): Sound

Cheap points, big demo impact. `pygame.mixer` is already available.

```
pygame.mixer.init()
step = pygame.mixer.Sound("assets/step.wav")
step.play()
```

Need: footstep, caught, win, and optionally a low pulse that gets faster as the enemy closes in (use `len(self.enemy.path)` as the distance — it's already computed).

Get free sounds from **freesound.org** or generate with **sfxr/jsfxr** (bfxr.net — click "Pickup", download, done). Keep files in `assets/`.

Guard every sound call so a missing file can't crash the demo:

```
try:
    pygame.mixer.init()
    SOUND_ON = True
except pygame.error:
    SOUND_ON = False
```

Part 3 (~1 hour): Testing — your most important job

`test_pathfinding.py` has 87 checks. Run it after *every* change anyone makes:

```
python test_pathfinding.py
```

Add tests for

- Player cannot walk through walls (call `try_move` into a wall, assert the cell didn't change)
- Move cooldown works (two moves in the same millisecond → only one happens)
- Win triggers when the player reaches the exit
- Loss triggers when the enemy reaches the player
- The game survives 200 random moves without crashing (fuzz test)

The fuzz test is the highest-value one

```
import random
game = Game()
for _ in range(200):
    d = random.choice([(0,-1),(1,0),(0,1),(-1,0)])
    game.player.try_move(game.grid, d, now)
    game.enemy.update(game.grid, game.player.cell, now)
```

This is how you find the crash that would otherwise happen *in front of the examiner*.

Breaking things on purpose

Try: holding two arrow keys at once, spamming R during the win banner, pressing TAB repeatedly mid-chase, ALT-TABbing away and back, resizing the window. Write down anything that misbehaves and tell the owner of that file.

Part 4 (~30 min): Screenshots for submission

The deliverables list requires sample output. There's a working headless screenshot script pattern — ask Akhil for `shot.py`, or capture live with a screen grab. Need at least:

- BFS overlay mid-chase
- A* overlay from the same position (the comparison shot)
- Win screen
- Lose screen

Save to `assets/`. Member D needs these for the slides, so get them done by hour 8 — don't leave it to the end.

What you must be able to answer

- How does the game loop work? (handle events → update state → draw → tick)
- What is the state machine and why use one?
- Why is movement timed in milliseconds instead of frames? (So the game plays the same on a fast and a slow machine.)
- What's the purple overlay showing? (Cells the search expanded.)
- How did you test it, and what bugs did you find?

7. Member D — Analysis & Docs

You own two things: the measurement tools, and the submission itself.

Your code is different from the others' — you're not building game features, you're building the tools that *prove the game works and measure how well*. That output goes straight into your own slides, so the two halves of your job feed each other.

You also hold the hackathon's single most important insurance policy: **the backup video**.

Priority order — do them in this order

1. Backup demo video — FIRST, done by hour 8

Not at hour 11. The rubric explicitly asks for it, and if the live demo crashes in front of the examiner, this video is the difference between 8/8 and 3/8 on **Functionality**.

- Record with OBS (free), Xbox Game Bar (`Win+G` , built into Windows), or any screen recorder.
- 3–4 minutes. Show: title → start → chase with overlay on → TAB to switch algorithm → win → next level.
- Narrate it, or add captions. Don't rely on the room being quiet.
- Save as `assets/demo_video.mp4` **and** upload to Drive with a shareable link in the README. If the laptop dies, you still have the link.
- **Re-record at hour 10** if features changed. The old one stays your fallback until the new one exists.

Get this done and the rest of your day is calm.

2. CODE: `benchmark.py` — the BFS vs A* measurement tool (~1.5 hours)

This is your main coding task. It's a **standalone script** — it imports the game's modules but never modifies them, so you cannot possibly break the demo. Safest code on the team to be writing.

What it must do

1. Generate N mazes (N = 50 or so) with different seeds.
2. For each, run BFS and A* from the player start to the exit.
3. Record: cells expanded by each, path length, time taken by each.
4. Print a summary table.
5. Save a bar chart to `assets/benchmark.png` using matplotlib (already installed).

6. Repeat the whole thing at several `braid_chance` values, so the chart shows how A*'s advantage changes as the maze opens up.

Starting points

Everything you need already exists:

```
import maze
from pathfinding import bfs, astar
from settings import COLS, ROWS

grid = maze.generate(seed=0, braid_chance=0.35)
path, visited = bfs(grid, (1, 1), (COLS - 2, ROWS - 2))
# len(path) = route length
# len(visited) = cells the search expanded <-- this is the headline number
```

For timing, use `time.perf_counter()`, not `time.time()` — it's the high-resolution one meant for measuring short durations.

```
import time
start = time.perf_counter()
bfs(grid, s, g)
elapsed_ms = (time.perf_counter() - start) * 1000
```

Hints for when you get stuck

- **Timings come out as 0.0** — a single search is too fast to measure. Run it 100 times in a loop and divide, or use `timeit`.
- **Results jump around between runs** — you're using random seeds. Pass a fixed `seed=` to `maze.generate()` so your numbers are reproducible. Examiners like reproducible.
- **matplotlib opens a window and blocks** — you don't want that in a script. Call `matplotlib.use("Agg")` before importing pyplot, then `plt.savefig()` instead of `plt.show()`.
- **Chart is unreadable** — label the axes, add a legend, and set `plt.tight_layout()` before saving.

Expected result

You should reproduce something close to the table already in the README:

Braid chance	BFS expanded	A* expanded	A* saving
0.00	167.6	152.2	9.2%
0.35	~221	~161	~27%
0.80	260.3	166.7	36.0%

If your numbers differ meaningfully, that's interesting, not a failure — find out why and say so. Different maze size, different start/goal, different sample count all change it.

Why this is worth doing

Most hackathon teams claim "A* is faster." You will have a chart with measured numbers across 50 mazes. That's the difference between an assertion and evidence, and it carries **Technical Implementation (6)** and **Innovation (3)**.

3. CODE: `autoplay.py` — the robot tester (~45 min)

You're the team's second tester. Automate it.

What it does

Runs the game headlessly with a bot playing it, hundreds of times, looking for crashes and impossible levels — the bugs that would otherwise surface in front of the examiner.

Core idea

```
import os
os.environ["SDL_VIDEODRIVER"] = "dummy"    # no window opens
from main import Game, WON, LOST
```

Then loop: pick a move for the player, advance the enemy, check the state.

The simplest bot moves randomly. A better one follows the BFS path *to the exit* — you already have `bfs()`, so the bot can use the same pathfinding the enemy does, just aimed at the exit instead of the player.

What to look for and report

- Any crash or exception — report immediately, with the seed that caused it
- Levels where the bot can *never* win, even playing optimally (the enemy is too fast — tell Member B)
- Levels that end instantly (enemy spawned too close — tell Member A)

Hints

- `pygame.time.get_ticks()` moves in real time, so the cooldowns will throttle your bot to real speed. Pass your own increasing `now` value instead to run thousands of steps instantly.
- Wrap each run in `try/except` and record the seed on failure, so a crash gives you a reproducible case rather than just "it broke once."

Why this matters

"We ran 500 automated playthroughs and found two levels that were unwinnable" is a genuinely strong line in the viva.

4. Team Details Sheet

Required deliverable. Fill in the table at the bottom of `README.md`: team name, every member's name and roll number, problem statement chosen (#15), and one line of contribution each.

Also: **Annexure B (SEE Examination Evaluation sheet)** must be filled in *in advance* by each member and signed before handing to the examiner. Chase your teammates — it's the easiest thing to forget and it's mandatory.

5. Slide deck — 3 to 5 slides, no more

Slide	Content
1	Problem statement #15, team name, members
2	Solution + screenshot of the game with the search overlay on
3	Tech stack and architecture — the modules and how they connect
4	Challenges faced (below) + your benchmark chart
5	Future scope (pull from the README)

Slide 4 is your strongest, and it's built from your own output.

6. README accuracy check — hour 10

`README.md` is already detailed. Your job is keeping it **true** as features change. If Member A adds a second enemy and the README doesn't mention it, that's a mark lost on **Code Quality & Documentation (2)**.

Verify on a clean machine: does `pip install -r requirements.txt && python main.py` actually work?

Your strongest slide: Challenges Faced

Judges hear "integration was hard" from every team. Specific bugs with specific fixes stand out. You already have three:

1. The enemy camped the exit. We first spawned the enemy at the cell farthest from the player. That's the opposite corner — exactly where the exit is. It sat on the goal and the level was unwinnable. Fixed by maximising the *smaller* of (distance to player, distance to exit), using true walking distance so walls count.

2. A* barely beat BFS. We expected A* to expand far fewer cells; it saved only 9%. In a tight maze the corridors are so constrained there's barely any choice for the heuristic to discriminate between. Opening ~35% of dead ends into loops raised A*'s saving to ~30% *and* made the game fairer by giving the player

escape routes.

3. Straight-line distance was the wrong measure. Manhattan distance calls two cells "close" when a wall between them makes the real walk 40 steps. We added a flood-fill measuring true walking distance and used that for enemy placement.

Put your benchmark chart next to these. Real numbers beat claims.

Stretch, only if everything above is done: collectibles

A small gameplay feature that's well within reach and visibly a *feature* to a judge: place 3 coins on random floor cells; the exit stays locked until all 3 are collected.

- `maze.floor_cells(grid)` gives you every valid position — pick 3 at random, away from the player start.
- Collected when the player's cell equals a coin's cell — same equality check the enemy catch already uses.
- Draw them as small circles; draw the exit differently while locked.
- Gate the win condition on `len(self.coins) == 0`.

This forces the player to cross the maze instead of beelining to the exit, which makes the chase matter much more.

What you must be able to answer

- What did you measure, and what did you find?
 - Why does A*'s advantage depend on the maze being open?
 - How many playthroughs did you automate, and what did they find?
 - How do you know the game is winnable at every level?
-

Hour 11.5 final checklist

- ☐ Code pushed to Git, repo accessible to the examiner
- ☐ `benchmark.py` runs and regenerates the chart
- ☐ `autoplay.py` runs clean
- ☐ README accurate, run instructions verified on a clean machine
- ☐ Slides done, exported to PDF (don't rely on PowerPoint opening)
- ☐ Backup video recorded **and** uploaded, link in README
- ☐ Screenshots in `assets/`
- ☐ Team details sheet filled
- ☐ Annexure B signed by all 4 members
- ☐ Everyone knows which part they're presenting

- ☐ Full demo rehearsed end to end, exactly as you'll present it
- ☐ Laptop charged, charger packed